CS143 Spring 2026 – Written Assignment 1

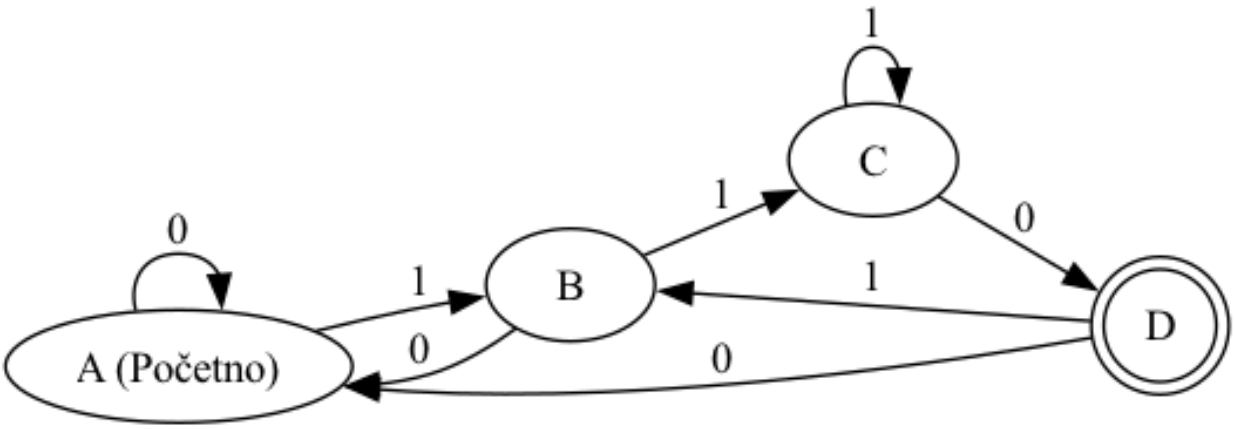
Due Thursday, April 16, 2026 11:59 PM PDT

This assignment covers regular languages, finite automata, and lexical analysis. You may discuss this assignment with other students and work on the problems together. However, your write-up should be your own individual work, and you should indicate in your submission who you worked with, if applicable. Assignments can be submitted electronically through Gradescope as a PDF by 11:59 PM PDT on the due date. Please review the course policies for more information: `http://web.stanford.edu/class/cs143/policies/index.html`. A LaTeX template for writing your solutions is available on the course website. To create finite automata diagrams, you can either use the Ti*k*Z package directly by following the examples in the template, or a tool like `https://madebyevan.com/fsm/`.

1. Write regular expressions *and* DFAs that recognize the following languages over the alphabet $\Sigma = \{0, 1\}$. Your DFAs must contain no more than 4 states.

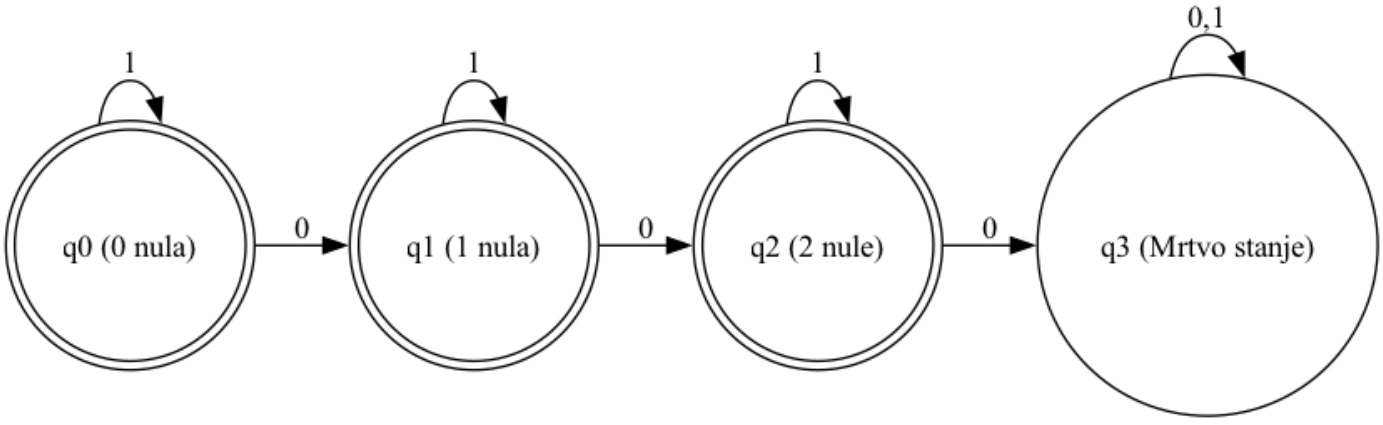
   (a) The set of strings that end with 110.

   Solution: (1|0)*110



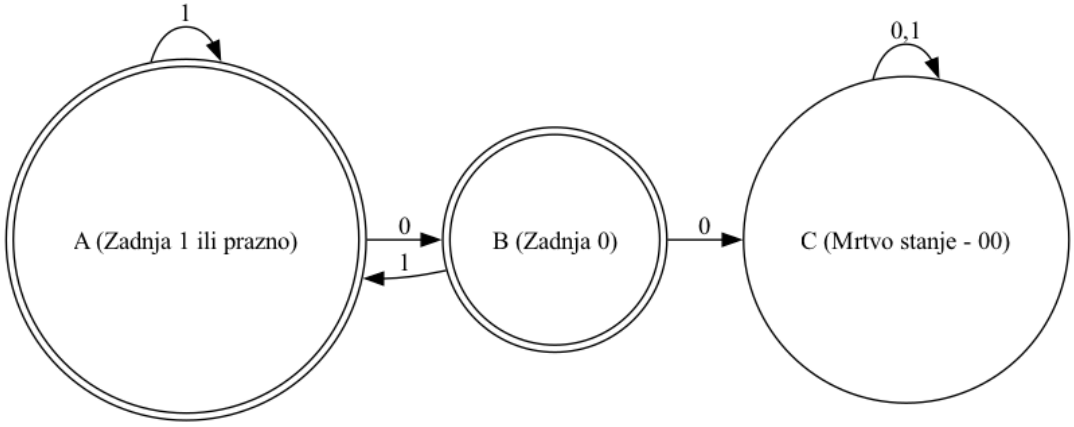
   (b) The set of strings that contain less than three 0's.

   Solution: 1*|1*01*|1*01*01*



(c) The set of strings that do not contain two or more consequent 0's.

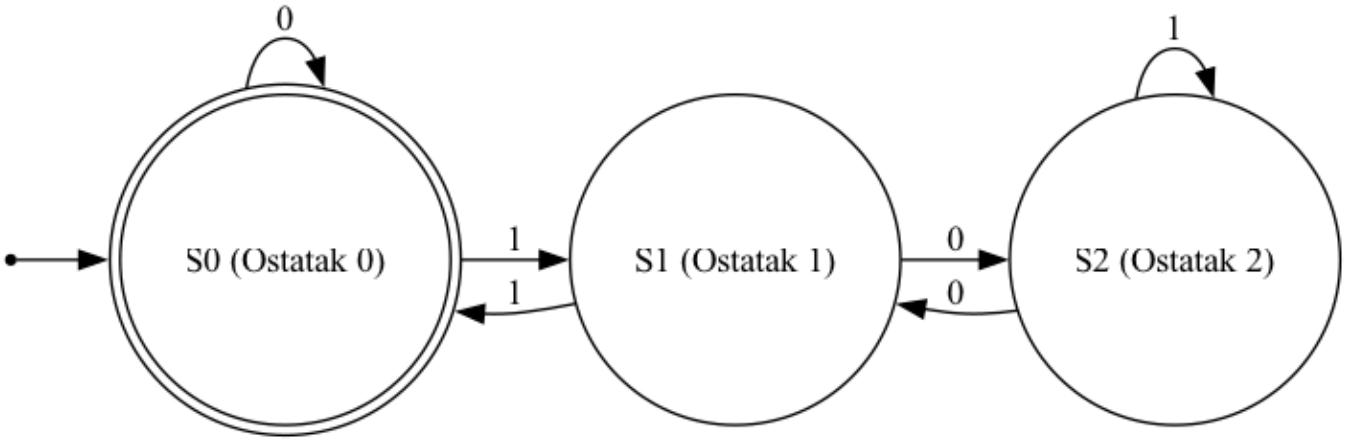
Solution: (1|01)*(0|ε)



(d) The set of strings that, when interpreted as a binary number, is a multiple of 3 (as an edge case, the empty string shall be interpreted as number 0, which is a multiple of 3). **Hint:** For this subproblem, it might be easier to draw the DFA first, and then construct the regular expression by considering what paths are possible in the DFA to reach the accept state.

Solution: (0|1(1|01*0)*1)*

Trenutno stanje	Ulaz 0	Ulaz 1	Ostatak / Značenje
S0 (Početno/Završno)	S0	S1	Ostatak 0 (Djeljivo s 3)
S1	S2	S0	Ostatak 1
S2	S1	S2	Ostatak 2

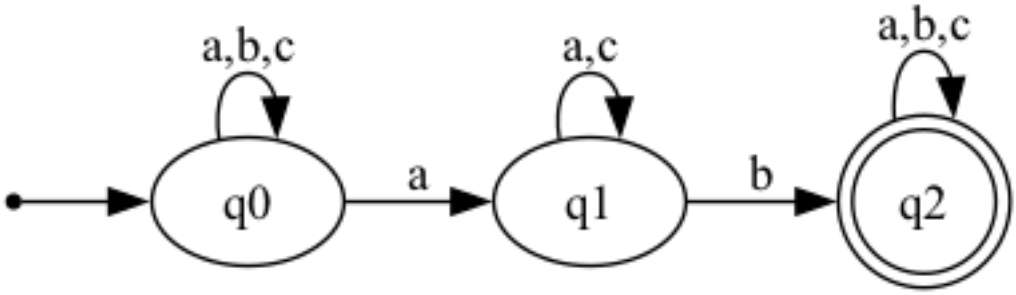


2. Write NFAs that recognize the following languages over the alphabet $\Sigma = \{a, b, c\}$.

   (a) The language L where a string w is in L iff there exists $i < j$ such that $w[i] =$`'a'` and $w[j] =$`'b'`. For example, `ab`, `bacb` are in L, where `ba`, `ac` are not. Your NFA should have no more than 3 states.

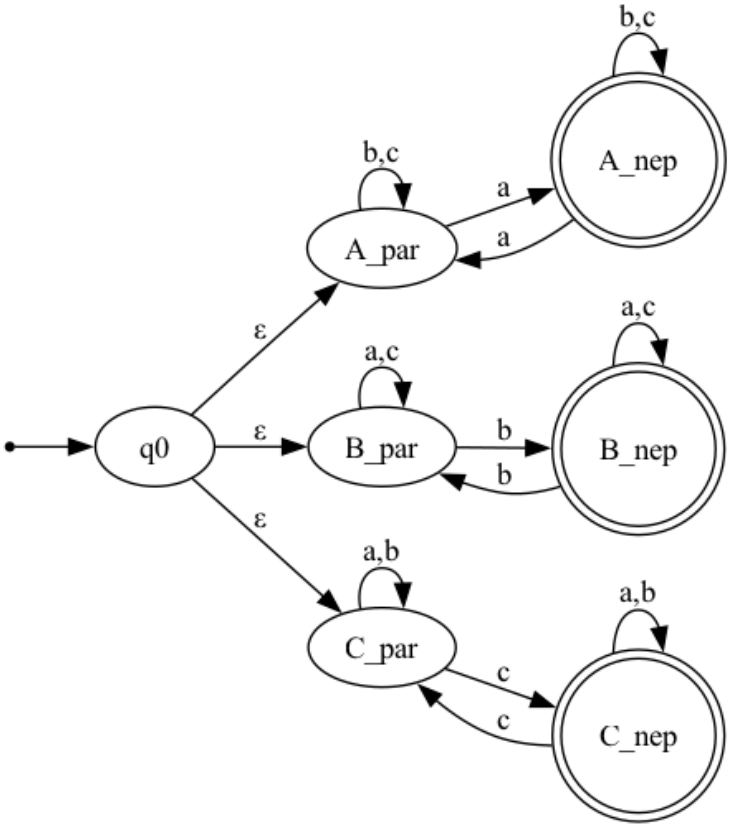
Solution: (a|b|c)*a(b|c)*b(a|b|c)*

Trenutno stanje	Ulaz a	Ulaz b	Ulaz c
q0 (Početno)	{q0, q1}	{q0}	{q0}
q1	{q1}	{q2}	{q1}
q2 (Završno)	{q2}	{q2}	{q2}



   (b) The language L where a string w is in L iff some character $x \in \{a, b, c\}$ appears an odd number of times in w. For example, `ab` is in L, but `aa` is not. Your NFA should have no more than 8 states.

Solution:



Koncept automata:
Svaka grana prati parnost jednog znaka i sastoji se od tačno 2 stanja:
Prvo stanje u grani - znak se pojavio paran broj puta.
Drugo stanje u grani - znak se pojavio neparan broj puta (Završno stanje). Ostala dva znaka u toj grani se ponašaju kao petlje (self-loops) i ne mijenjaju stanje parnosti.

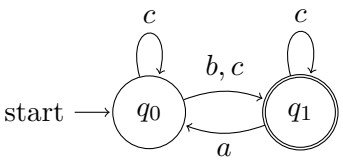
Stanje	Ulaz a	Ulaz b	Ulaz c	Ulaz ε
q0 (Početno)	Ø	Ø	Ø	{A_par, B_par, C_par}
A_par	{A_nep}	{A_par}	{A_par}	Ø
A_nep (Završno)	{A_par}	{A_nep}	{A_nep}	Ø
B_par	{B_par}	{B_nep}	{B_par}	Ø
B_nep (Završno)	{B_nep}	{B_par}	{B_nep}	Ø
C_par	{C_par}	{C_par}	{C_nep}	Ø
C_nep (Završno)	{C_nep}	{C_nep}	{C_par}	Ø

3. Using the techniques covered in class, transform the following NFAs over the alphabet $\{a, b, c\}$ into DFAs. Your DFAs should not have more than 10 states. Note that a DFA must have a transition defined for every state and symbol pair, whereas a NFA need not. You must take this fact into account for your transformations. Hint: Is there a subset of states the NFA transitions to when fed a symbol for which the set of current states has no explicit transition?

   Also include a mapping from each state of your DFA to the corresponding states of the original NFA. Specifically, a state q of your DFA maps to the set of states Q of the NFA such that an input string stops at q in the DFA if and only if it stops at one of the states in Q in the NFA.

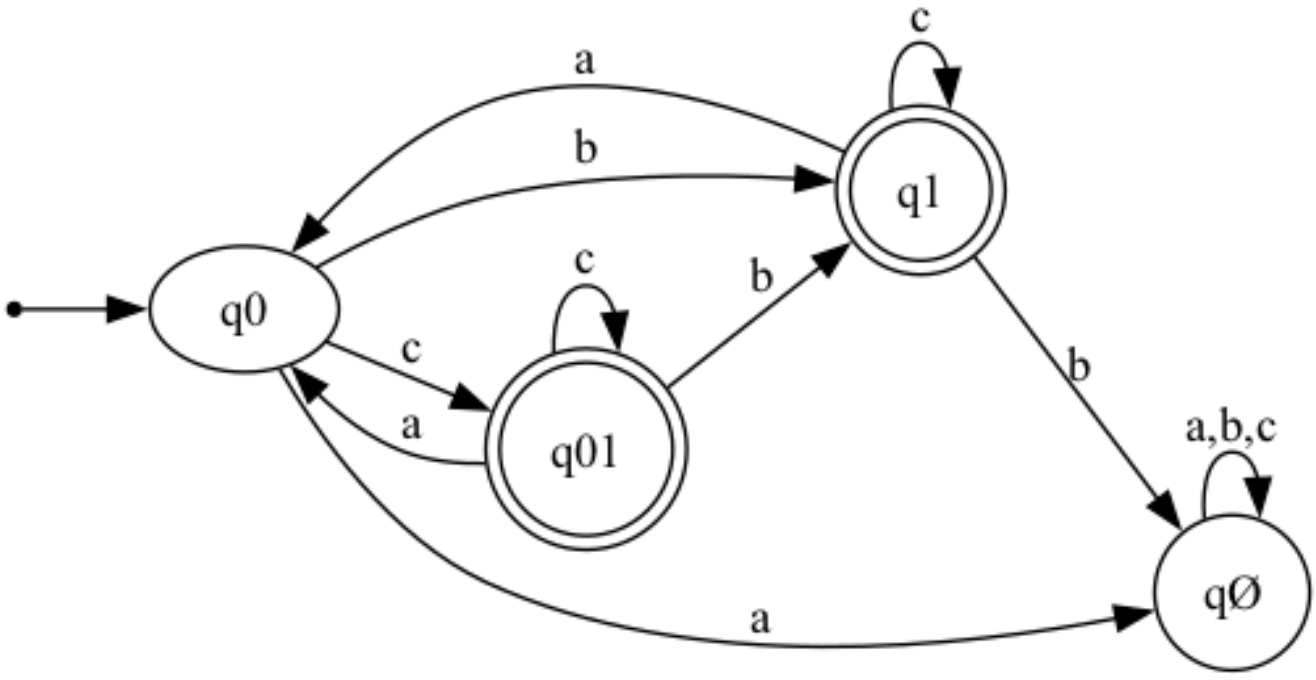
   Tip: for readability, states in the DFA may be labeled according to the set of states they represent in the NFA. For example, state q_{012} in the DFA would correspond to the set of states $\{q_0, q_1, q_2\}$ in the NFA, whereas state q_{13} would correspond to set of states $\{q_1, q_3\}$ in the NFA.

   (a)

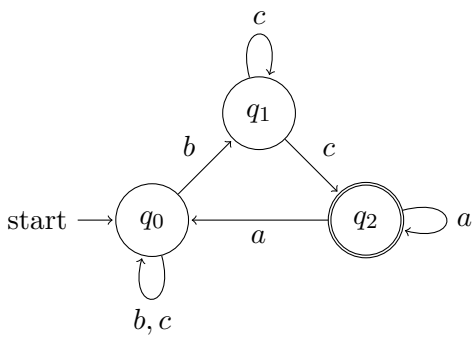


   Solution:

Trenutno stanje	Ulaz a	Ulaz b	Ulaz c
q0 (Početno)	qØ	q1	q01
q1 (Završno)	q0	qØ	q1
q01 (Završno)	q0	q1	q01
qØ (Mrtvo stanje)	qØ	qØ	qØ

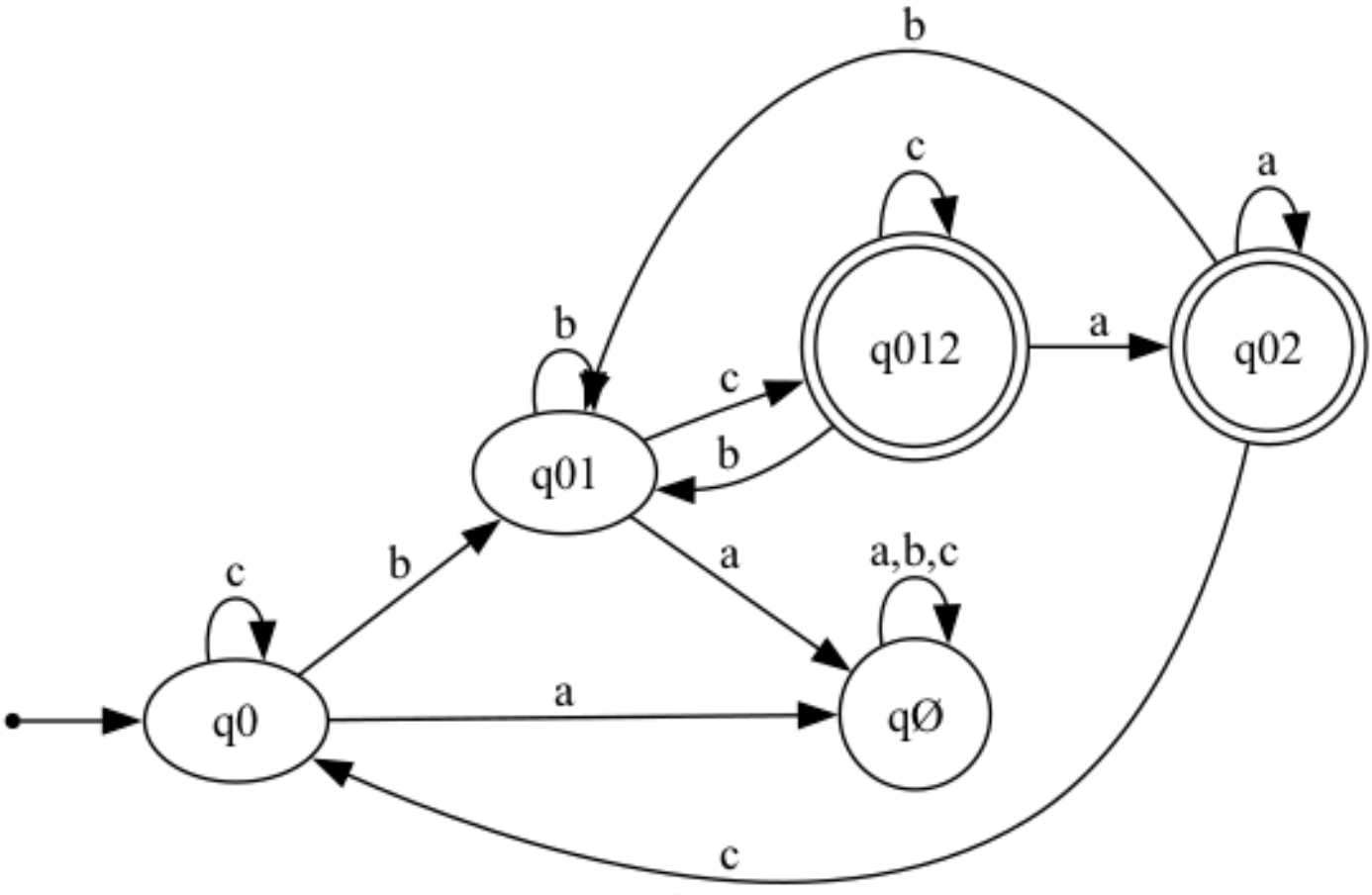


(b)

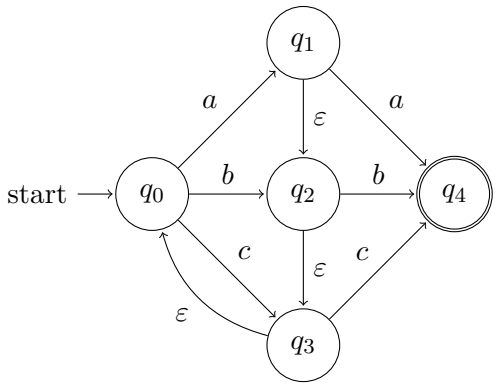


Solution:

Trenutno stanje	Ulaz a	Ulaz b	Ulaz c
q0 (Početno)	qØ	q01	q0
q01	qØ	q01	q012
q012 (Završno)	q02	q01	q012
q02 (Završno)	q02	q01	q0
qØ (Mrtvo stanje)	qØ	qØ	qØ

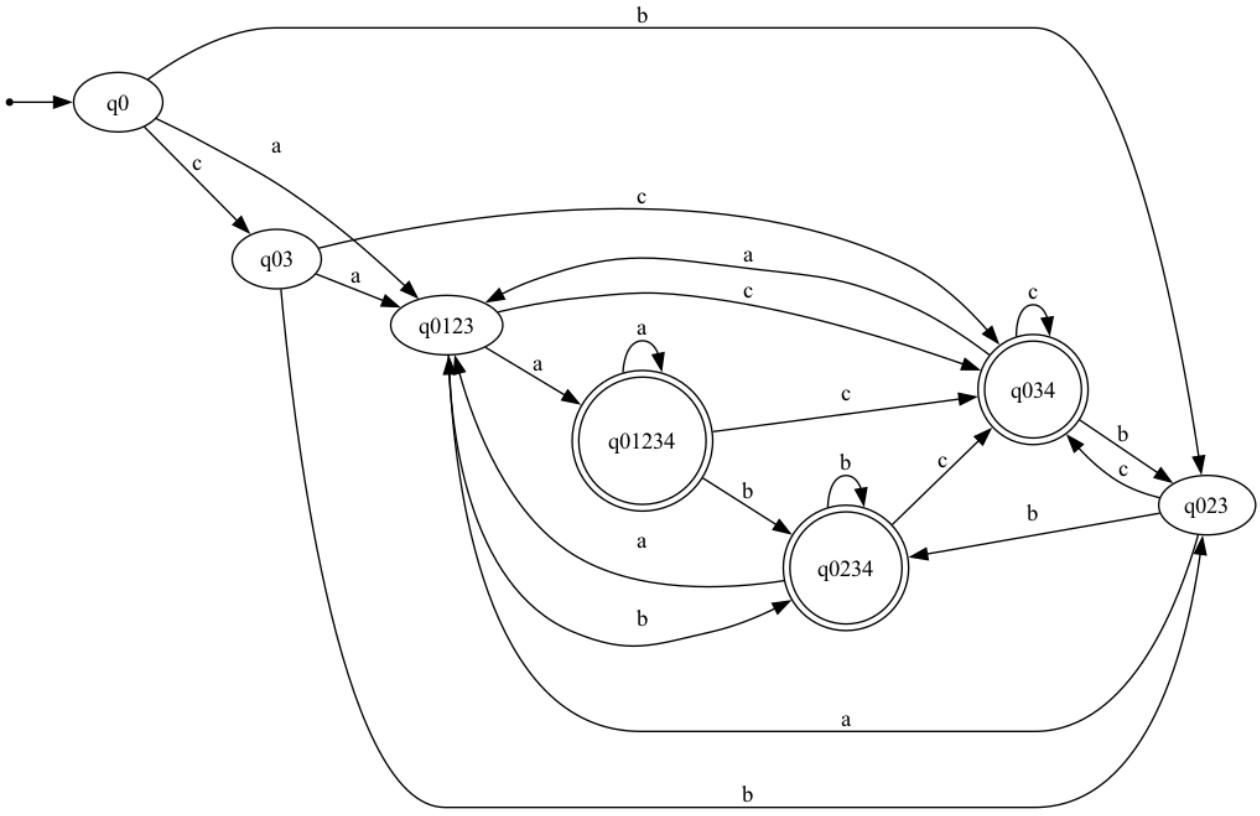


(c)



Solution:

Trenutno stanje	Ulaz a	Ulaz b	Ulaz c
q0 (Početno)	q0123	q023	q03
q0123	q01234	q0234	q034
q023	q0123	q0234	q034
q03	q0123	q023	q034
q01234 (Završno)	q01234	q0234	q034
q0234 (Završno)	q0123	q0234	q034
q034 (Završno)	q0123	q023	q034



4. Consider the following tokens and their associated regular expressions, given as a **flex** scanner specification:

```
%%
0                       printf("a");
(10|01)                 printf("b");
(101|011+)              printf("c");
```

Give an input to this scanner such that the output lexeme sequence is `aacbbca`.

For ease of grading, please use underlines to show the parts of your input string that correspond to each lexeme, for example, 10 0 0111.

Solution:

Ulazni niz (string) koji daje traženi izlaz **aacbbca** je:

0 0 011 01 01 011 0

Kako funkcioniše

Prilikom dizajniranja ulaza za **flex** skener, moramo uzeti u obzir pravilo **"Maximal Munch"** (pravilo najdužeg poklapanja): skener će uvijek pokušati da upari najduži mogući niz karaktera za neki token. Jednostavno nadovezivanje ispravnih regularnih izraza često ne uspijeva jer se granični karakteri stapaju u duža poklapanja i prave grešku.

Evo analize korak po korak kako skener čita ovaj niz:

- `a` → **0**: Skener čita prvu `0`. Gleda unaprijed sljedeći karakter (koji je isto `0`), ali `00` ne odgovara nijednom pravilu. Zato se zaustavlja i sigurno ispisuje `a`.
- `a` → **0**: Skener čita sljedeću `0`. Gleda unaprijed u `011`, ali `001` nije pravilo. Opet se bezbjedno zaustavlja i ispisuje `a`.
- `c` → **011**: Ovdje **moramo** koristiti `011` umjesto `101`. *(Da smo iskoristili `101`, niz bi do tog trenutka bio `00101`, a skener bi onu drugu nulu grupisao sa jedinicom pored nje i ispisao `b` za token `01`, čime bi uništio traženi redoslijed).* Skener čita `011` i vidi da je sljedeći karakter `0`. Pošto `0110` ne odgovara ničemu, ispisuje `c`.
- `b` → **01**: Ovdje **moramo** koristiti `01` umjesto `10`. *(Da smo iskoristili `10`, niz bi izgledao kao `...01110...`. Skener bi pohlepno progutao tu dodatnu jedinicu i dodao je u prethodni `c` token zbog pravila `011+`, što bi rezultiralo ispisom `c`, a onda bi za nulu ispisao `a`.)* Skener uparuje `01` i sigurno ispisuje `b`.
- `b` → **01**: Ponovo moramo koristiti `01`. *(Da smo koristili `10`, niz bi se spojio u `...0110...`, a skener bi od toga napravio samo jedno `c` zbog pravila `011`).* Skener uparuje `01` i ispisuje `b`.
- `c` → **011**: Skener čita `011`. Gleda posljednji karakter (`0`) i zna da mora stati jer `0110` nije ispravno poklapanje. Ispisuje `c`.
- `a` → **0**: Posljednji karakter je samo jedna nula koja se uparuje po prvom pravilu, ispisujući `a`.

5. Recall from the lecture that, when using regular expressions to scan an input, we resolve conflicts by taking the largest possible match at any point. That is, if we have the following **flex** scanner specification:

```
%%
do                          { return T_Do; }
[A-Za-z_][A-Za-z0-9_]*      { return T_Identifier; }
```

and we see the input string "`dot`", we will match the second rule and emit T_Identifier for the whole string, not T_Do.

However, it is possible to have a set of regular expressions for which we can tokenize a particular string, but for which taking the largest possible match will fail to break the input into tokens. Give an example of no more than two regular expressions and an input string such that: a) the string can be broken into substrings, where each substring matches one of the regular expressions, b) our usual lexer algorithm, taking the largest match at every step, will fail to break the string in a way in which each piece matches one of the regular expressions. Explain how the string can be tokenized and why taking the largest match won't work in this case.

As a challenge (not necessary for credit), try to find a solution that only uses one regular expression.

Solution:

Zašto Greedy matching zakazuje

Teoretski ispravna podjela (aab = a + ab):

a ab

Reg. izraz: (a+|ab) | Ulazni string: 'aab'

1. Greedy korak (Pozicija 0): Algoritam traži najduži meč.
   'a+' matchuje 'aa', dok 'ab' ne matchuje (jer su prva dva slova 'aa').
   Maximal Munch uzima 'aa' kao prvi token.

2. Ostatak stringa: Na traci ostaje 'b'.

3. Fatalna greška: Skener pokušava matchovati 'b' pravilima 'a+' i 'ab'.
   Nijedno ne odgovara, pa lekser prijavljuje grešku i puca.

Suština: Pohlepni algoritam je pojeo jedno 'a' previše, a to 'a'
je bilo neophodno kao 'sidro' za početak drugog tokena 'ab'.